

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Майстренко Анастасия Николаевна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать REST API серверной части сервиса аренды недвижимости на основе подходящего boilerplate. На основании результатов Д31 (проектирование БД) и Д32 (проектирование API) необходимо реализовать:

- модели (сущности предметной области);
- представления (сериализация данных и маршруты);
- контроллеры (обработка запросов).

Результат — полностью работающее API согласно выбранному варианту (Сервис для аренды недвижимости).

Ход работы

1. Выбор boilerplate и стека

В качестве основы взят boilerplate `express-typeorm-boilerplate`, предложенный преподавателем. Он построен на следующем стеке:

- Node.js + TypeScript — среда выполнения и язык;
- Express + routing-controllers — веб-сервер и контроллеры на основе декораторов;
- TypeORM — ORM для описания моделей и работы с БД (режим `synchronize` автоматически создаёт схему);
- PostgreSQL — основная СУБД (поднимается через `docker-compose`);
- `class-validator` / `class-transformer` — валидация и преобразование тел запросов;
- `jsonwebtoken` и `bcrypt` — JWT-аутентификация и хеширование паролей;
- `routing-controllers-openapi` + `swagger-ui-express` — автогенерация OpenAPI-документации и Swagger UI на `/docs`.

2. Структура проекта

Проект разделён на слои в соответствии с архитектурой boilerplate:

<code>src/models/</code>	— модели (сущности TypeORM)
<code>src/dto/</code>	— DTO-классы для валидации запросов
<code>src/controllers/</code>	— контроллеры и маршруты (представления)
<code>src/middlewares/</code>	— middleware аутентификации (JWT)
<code>src/common/</code>	— базовый контроллер и декоратор
<code>EntityController</code>	
<code>src/config/</code>	— настройки и подключение к БД
<code>src/utils/</code>	— хеширование, выдача токенов, пагинация
<code>src/app.ts</code>	— точка входа, <code>swagger.ts</code> — документация

3. Модели (models)

Реализованы модели, соответствующие ER-диаграмме из ДЗ1. Каждая модель — класс TypeORM с декораторами @Entity, @Column и описанием связей (@ManyToOne, @OneToMany, @ManyToMany). Перечень моделей приведён в таблице 1.

Таблица 1 — Модели предметной области

Модель	Назначение и связи
User	Пользователь; роль tenant/landlord/admin; владеет Property, делает Booking, имеет избранное (M:N с Property)
Property	Объект недвижимости; принадлежит User; имеет фото, удобства (M:N), бронирования
PropertyPhoto	Фотография объекта (1:N от Property)
Amenity	Справочник удобств; M:N с Property через таблицу property_amenity
Booking	Сделка аренды; связана с Property и арендатором User; имеет статус
Review	Отзыв по завершённой сделке (Booking) от автора (User)
Conversation	Диалог двух пользователей, опционально по объекту
Message	Сообщение в диалоге (1:N от Conversation), отправитель User

Хеширование пароля вынесено в TypeORM-subscriber (user.subscriber.ts): перед вставкой и обновлением пароль автоматически хешируется bcrypt, поэтому в БД он никогда не хранится в открытом виде.

4. Контроллеры и представления (controllers, views)

Контроллеры реализованы на декораторах routing-controllers. Каждый контроллер через собственный декоратор @EntityController получает доступ к репозиторию своей сущности (this.repository). Представления — это методы контроллеров, которые принимают валидированные DTO, обращаются к репозиториям и возвращают JSON. Перечень контроллеров — в таблице 2.

Таблица 2 — Контроллеры и реализованная функциональность

Контроллер	Функциональность
AuthController	Регистрация, вход (выдача JWT), обновление токенов
UserController	Профиль (/me), публичный профиль, ЛК: сдаваемые и арендуемые объекты, избранное
PropertyController	Поиск с фильтрами, CRUD объекта, фото, отзывы по объекту
AmenityController	Справочник удобств (список, добавление)
BookingController	Создание брони с расчётом стоимости, история сделок, смена статуса, отзыв
ConversationController	Диалоги, отправка и история сообщений, отметка о прочтении

Поиск недвижимости реализован через QueryBuilder с фильтрами по типу, цене, городу и числу комнат, а также пагинацией и сортировкой. Списочные ответы возвращаются в едином формате $\{ items, meta \}$, где meta содержит сведения о пагинации.

5. Аутентификация и контроль доступа

Аутентификация выполнена на JWT. При входе и регистрации выдаётся пара токенов (access и refresh). Защищённые маршруты используют middleware authMiddleware, который проверяет заголовок Authorization: Bearer <token>, и при невалидном или отсутствующем токене возвращает 401. Контроль доступа реализован на уровне контроллеров: например, редактировать и удалять объект может только его владелец, а читать переписку — только участники диалога (иначе 403).

6. Проверка работоспособности

API было запущено и протестировано полным набором сценариев (запросы curl к запущенному серверу). Результаты приведены в таблице 3 — все сценарии отработали корректно, включая бизнес-правила и проверки доступа.

Таблица 3 — Результаты проверки API

Сценарий	Ожид.	Факт
Регистрация пользователя	201	201
Вход с верным паролем	200	200
Вход с неверным паролем	401	401
Создание объекта без токена	401	401
Создание объекта с некорректным телом (валидация)	400	400
Создание объекта владельцем	201	201
Поиск с фильтрами по типу и цене	200	200
Страница объекта (фото, удобства, рейтинг)	200	200
Бронирование: расчёт стоимости $5 \times 2500 = 12500$	201	201
Бронирование с датой конца раньше начала	400	400
Отзыв до завершения сделки	400	400
Отзыв после завершения сделки	201	201
ЛК: список арендуемых / сдаваемых объектов	200	200
Избранное: добавить / получить / удалить	204/200	204/200
Сообщения: диалог, отправка, история, прочтение	201/200/204	ОК
Чтение чужого диалога посторонним	403	403
Swagger UI (/docs)	200	200

Для локального запуска без внешней БД предусмотрен режим SQLite (DB_TYPE=sqlite); основной режим работы — PostgreSQL через docker-compose. Документация API доступна в Swagger UI по адресу /docs.

Вывод

В ходе лабораторной работы на основе boilerplate express-typeorm-boilerplate реализовано полноценное REST API сервиса аренды недвижимости. Спроектированные ранее (Д31, Д32) сущности и эндпоинты воплощены в виде моделей TypeORM, DTO-представлений и контроллеров routing-controllers.

Реализованы все ключевые сценарии варианта: регистрация и аутентификация, личный кабинет со списком арендуемых и сдаваемых объектов, поиск с фильтрацией, страница объекта, бронирование и история сделок, отзывы, избранное и обмен сообщениями. Добавлены валидация входных данных, JWT-аутентификация, хеширование паролей и контроль доступа по владельцу/участнику. Работоспособность API подтверждена набором тестовых запросов; документация доступна через Swagger UI.